🎮 GAME CAFE SYSTEM — TECHNICAL DOCUMENTATION

คู่มือฐานข้อมูลและสถาปัตยกรรมระบบ
Database Schema & Architecture Guide

เอกสารประกอบการนำเสนอฐานข้อมูล สำหรับอาจารย์ผู้ดูแล

ชื่อโปรเจกต์	เทคโนโลยี	จำนวนตารางหลัก
LETSPLAY GAMING — ระบบบริหารร้านเกมคาเฟ่	PHP 8.4 · Laravel 12 · SQLite	11 Tables

ฐานข้อมูล
database/database.sqlite

1 ภาพรวมระบบ (System Overview)

ผู้ใช้และกระเป๋าเงิน **users · wallet_transactions**	โซน เครื่อง และเซสชัน **zones · seats · seat_sessions**	แพ็กเกจชั่วโมง **packages · user_packages**
อาหารและออเดอร์ **categories · products · orders · order_items**	บทบาทผู้ใช้ **Admin · Staff · Customer**	ช่องทางชำระเงิน **Wallet · PromptPay · Cash**

สรุปสั้น: ระบบ Web Application บริหารร้านเกมคาเฟ่ครบวงจร ตั้งแต่จัดการสมาชิก, กระเป๋าเงินดิจิทัล, ตรวจสอบที่นั่ง (Seat Map), แพ็กเกจชั่วโมงล่วงหน้า และสั่งอาหารจากหน้าจอเครื่อง โดยใช้ SQLite เป็น Database

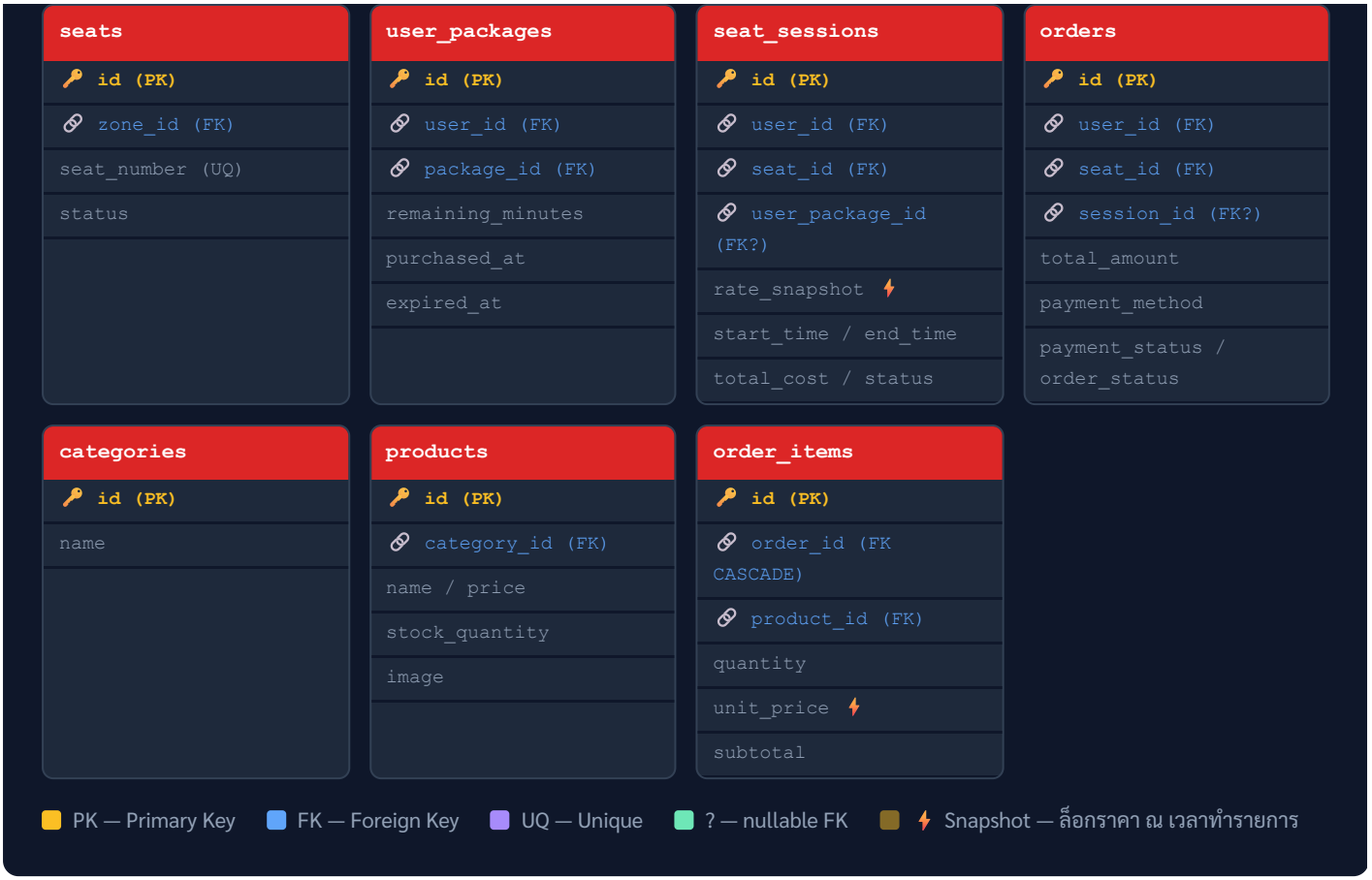
2 ผังความสัมพันธ์ตาราง (Entity Relationship Diagram)

users
🔑 id (PK)
name
username (UQ)
email (UQ)
role
balance
password

wallet_transactions
🔑 id (PK)
🔗 user_id (FK)
type
amount
ref_type
ref_id

zones
🔑 id (PK)
name
hourly_rate
description

packages
🔑 id (PK)
🔗 zone_id (FK)
name
duration_hours
price



ความสัมพันธ์หลัก: **users** → seat_sessions → orders → order_items | zones → seats → seat_sessions | packages → user_packages → seat_sessions

3 รายละเอียดโครงสร้างตาราง (Table Schema)

กลุ่มที่ 1 — ผู้ใช้งานและกระเป๋าเงิน

users บัญชีผู้ใช้งานทั้งหมด (Customer, Staff, Admin)

Column	Type	Constraint	คำอธิบาย
id	bigint	PK	รหัสผู้ใช้งาน (Auto Increment)
name	varchar(255)		ชื่อ-นามสกุล
username	varchar(50)	UQ	ชื่อบัญชีล็อกอิน (ห้ามซ้ำ)
email	varchar(255)	nullable UQ	อีเมล (ไม่บังคับ)
phone	varchar(20)	nullable	เบอร์โทรศัพท์
role	varchar(20)	default: customer	customer / staff / admin
balance	decimal(10,2)	default: 0.00	ยอดเงินในกระเป๋าดิจิทัล (บาท)
password	varchar(255)		รหัสผ่าน Bcrypt Hash

Relationships:
→ hasMany wallet_transactions / user_packages / seat_sessions / orders

wallet_transactions Audit Log บันทึกทุกการเปลี่ยนแปลงยอดเงิน

Column	Type	Constraint	คำอธิบาย
id	bigint	PK	รหัสธุรกรรม
user_id	bigint	FK→users	เจ้าของบัญชี
type	varchar(20)		topup เติมเงิน · deduct หักเงิน · refund คืนเงิน
amount	decimal(10,2)		จำนวนเงิน (บาท)
ref_type	varchar(50)	nullable	session / order / cash_topup / qr_topup / package_purchase
ref_id	bigint	nullable	ไอดีอ้างอิงจากตารางต้นทาง

→ belongsTo users

กลุ่มที่ 2 — โซน เครื่องคอมพิวเตอร์ และเซสชัน

zones โซนเครื่องในร้าน

Column	Type	คำอธิบาย
id	bigint	PK
name	varchar(100)	ชื่อโซน
hourly_rate	decimal(10,2)	ราคา/ชั่วโมง
description	text	รายละเอียดสเปก (nullable)

→ hasMany seats / packages

seats เครื่องคอมพิวเตอร์แต่ละเครื่อง

Column	Type	คำอธิบาย
id	bigint	PK
zone_id	bigint	FK→zones
seat_number	varchar(20)	UQ (STD-01, VIP-02)
status	varchar(20)	available / occupied / maintenance

→ belongsTo zones · hasMany seat_sessions / orders

seat_sessions รอบการใช้งานเครื่อง Check-in ถึง Check-out

Column	Type	Constraint	คำอธิบาย
id	bigint	PK	รหัสเซสชัน
user_id	bigint	FK→users	ลูกค้าที่ใช้งาน
seat_id	bigint	FK→seats	เครื่องที่ใช้งาน
user_package_id	bigint	FK→user_packages nullable	แพ็กเกจที่ใช้ (null = Pay As You Go)
rate_snapshot	decimal(10,2)		ราคา/ชั่วโมง ล็อกไว้ ณ ตอนเปิดเครื่อง
start_time	timestamp		เวลาเริ่มใช้
end_time	timestamp	nullable	เวลาสิ้นสุด (null = กำลังใช้อยู่)
total_cost	decimal(10,2)	default: 0.00	ค่าที่หักจริง (0 ถ้าใช้แพ็กเกจ)
status	varchar(20)	default: active	active กำลังเล่น · completed จบแล้ว

→ belongsTo users / seats / user_packages · hasMany orders

กลุ่มที่ 3 — แพ็กเกจชั่วโมง

packages แพ็กเกจที่ร้านเปิดขาย

Column	Type	คำอธิบาย
id	bigint	PK
zone_id	bigint	FK→zones (nullable = ทุกโซน)
name	varchar(100)	ชื่อแพ็กเกจ
duration_hours	integer	จำนวนชั่วโมง
price	decimal(10,2)	ราคาขาย (บาท)

user_packages แพ็กเกจที่ลูกค้าซื้อไว้ (Rich Pivot)

Column	Type	คำอธิบาย
id	bigint	PK
user_id	bigint	FK→users
package_id	bigint	FK→packages
remaining_minutes	integer	นาทีที่เหลือ
purchased_at	timestamp	วันที่ซื้อ
expired_at	timestamp	วันหมดอายุ (nullable)

กลุ่มที่ 4 — อาหาร เครื่องดื่ม และออเดอร์

categories หมวดหมู่อาหาร

Column	Type	คำอธิบาย
id	bigint	PK
name	varchar(100)	ชื่อหมวดหมู่ (เครื่องดื่ม, อาหารจานเดียว ฯลฯ)

→ hasMany products

products เมนูอาหารและสต็อก

Column	Type	คำอธิบาย
id	bigint	PK
category_id	bigint	FK→categories
name	varchar(150)	ชื่อสินค้า
price	decimal(10,2)	ราคาต่อหน่วย
stock_quantity	integer	สต็อกคงเหลือ (ตัดอัตโนมัติเมื่อสั่ง)
image	varchar(255)	path รูปภาพ (nullable)

orders คำสั่งซื้ออาหารและเครื่องดื่ม

Column	Type	Constraint	คำอธิบาย
id	bigint	PK	รหัสออเดอร์
user_id	bigint	FK→users	ลูกค้าที่สั่ง
seat_id	bigint	FK→seats	เครื่องที่ต้องการให้ส่ง
session_id	bigint	FK→seat_sessions nullable	เซสชันที่กำลังใช้
total_amount	decimal(10,2)	default: 0.00	ยอดรวม
payment_method	varchar(20)		wallet / promptpay / cash
payment_status	varchar(20)	default: pending	pending / paid / cancelled
order_status	varchar(20)	default: pending	pending → preparing → served / cancelled

→ belongsTo users / seats / seat_sessions · hasMany order_items

order_items รายการสินค้าย่อยในออเดอร์ (Line Items)

Column	Type	Constraint	คำอธิบาย
id	bigint	PK	รหัสรายการย่อย
order_id	bigint	FK→orders CASCADE	ลบออเดอร์แล้วรายการย่อยลบตาม
product_id	bigint	FK→products	สินค้าที่สั่ง
quantity	integer		จำนวน
unit_price	decimal(10,2)		ราคา ณ เวลาสั่ง (Snapshot ป้องกันราคาเปลี่ยน)
subtotal	decimal(10,2)		quantity × unit_price

→ belongsTo orders / products

4 ขั้นตอนการทำงาน (System Workflow & Data Flow)

CUSTOMER JOURNEY — ภาพรวมขั้นตอน



🔄 Workflow 1 — เติมเงิน (Top-up)

1. Staff รับเงินสดหรือยืนยัน QR แล้วเลือกลูกค้าในระบบ
2. ตรวจสอบจำนวน: ขั้นต่ำ **20 บาท** , สูงสุดต่อครั้ง **10,000 บาท**
3. อัปเดต `users.balance` += จำนวนที่เติม
4. บันทึก `wallet_transactions` : type = `topup` , ref_type = `cash_topup` หรือ `qr_topup`
5. ทุกขั้นตอนอยู่ใน `DB::transaction()` — ล้มเหลวตรงไหน Rollback ทั้งหมด

📦 Workflow 2 — ซื้อแพ็กเกจ (Buy Package)

1. ลูกค้าเลือกแพ็กเกจจากหน้าเว็บ (ดึงจาก `packages`)
2. ตรวจสอบ `users.balance` >= `packages.price`
3. หักเงิน: `users.balance` -= price
4. สร้าง record ใน `user_packages` : remaining_minutes = duration_hours × 60
5. บันทึก `wallet_transactions` : type = `deduct` , ref_type = `package_purchase`

🖥️ Workflow 3 — เปิดเครื่อง (Open Session)

1. เลือกเครื่องจาก Seat Map — ดึง `seats` ที่ status = `available`
2. เลือกโหมด: **A) Pay As You Go** (ต้องมี balance ≥ 20 บาท) หรือ **B) ใช้แพ็กเกจ**
3. สร้าง `seat_sessions` : status = `active` , start_time = now(), rate_snapshot = คัดลอกจาก `zones.hourly_rate` ทันที ⚡
4. อัปเดต `seats.status` = `occupied`
5. ถ้าใช้แพ็กเกจ: เชื่อม `seat_sessions.user_package_id` กับ `user_packages`

Workflow 4 — สั่งอาหาร (Place Order)

1. ลูกค้าเลือกเมนูจากหน้า Order (ดึง `products` + `categories`)
2. ตรวจสอบ `products.stock_quantity` เพียงพอ
3. ตรวจสอบ `users.balance` >= ยอดรวม
4. หักเงิน `users.balance` และตัดสต็อก `products.stock_quantity`
5. สร้าง `orders` : payment_status = `paid` , order_status = `pending`
6. สร้าง `order_items` : unit_price = คัดลอกราคาสินค้า ณ เวลานี้
7. บันทึก `wallet_transactions` : type = `deduct` , ref_type = `order`

Workflow 5 — จบเซสชัน (End Session)

1. Staff กด Check-out หรือระบบตัดอัตโนมัติ
2. คำนวณเวลาที่ใช้: end_time - start_time (นาที)
3. **Pay As You Go:** ค่าใช้จ่าย = (นาทีที่ใช้ / 60) × rate_snapshot → หัก `users.balance` → บันทึก `wallet_transactions`
4. **ใช้แพ็กเกจ:** หัก `user_packages.remaining_minutes` -= นาทีที่ใช้, total_cost = 0
5. อัปเดต `seat_sessions` : status = `completed` , end_time = now()
6. อัปเดต `seats.status` = `available`

วงจรสถานะออเดอร์อาหาร (ORDER STATUS LIFECYCLE)



Staff จัดการสถานะผ่านหน้า Kitchen Management | ยกเลิกออเดอร์จะ Rollback เงินคืนผ่าน `wallet_transactions` type = `refund`

5 เหตุผลในการออกแบบ (Design Rationale)

⚡ 1. Snapshot Pattern — ป้องกันราคาเปลี่ยนย้อนหลัง

ปัญหา: หากร้านปรับราคาโซนหรือเมนูอาหาร ยอดบิลเก่าที่บันทึกไว้จะผิดพลาด

วิธีแก้: คัดลอกราคา ณ เวลาที่ทำรายการเก็บไว้ในคอลัมน์แยก:

- `seat_sessions.rate_snapshot` — จาก zones.hourly_rate ตอนเปิดเครื่อง
- `order_items.unit_price` — จาก products.price ตอนสั่งอาหาร

ผล: ประวัติทางการเงินทุกรายการถูกต้อง แม้ราคาปัจจุบันจะเปลี่ยนไปแล้ว

🔒 2. Foreign Key Constraints — ป้องกัน Orphan Records

รูปแบบ	ใช้กับ	ผลที่เกิดขึ้น
RESTRICT	ความสัมพันธ์ส่วนใหญ่	ห้ามลบข้อมูลต้นทางที่มีลูกผูกอยู่ → ป้องกันสูญเสียประวัติการเงิน
CASCADE	orders → order_items	ลบออเดอร์แล้วรายการย่อยลบตาม → ไม่มีเศษข้อมูลค้างอยู่

🏦 3. ACID Transaction — การันตีความสมบูรณ์

ทุกรายการที่แตะข้อมูลมากกว่า 1 ตารางถูกห่อด้วย `DB::transaction()`

- **Atomicity:** สำเร็จทั้งหมด หรือ Rollback ทั้งหมด — ไม่มีสถานะค้างกลางทาง
- **Consistency:** users.balance ต้องตรงกับผลรวมใน wallet_transactions เสมอ
- **Isolation:** แต่ละรายการไม่รบกวนกัน
- **Durability:** เมื่อ commit แล้วข้อมูลบันทึกถาวร

📊 4. Rich Pivot Table (user_packages) — ไม่ใช่ Pivot ธรรมดา

ปัญหา: Pivot Table ทั่วไปไม่สามารถบันทึก "เวลาที่เหลือ" หรือ "วันหมดอายุ" ต่อ Instance ได้ และซื้อซ้ำไม่ได้

วิธีแก้: `user_packages` มี id เป็นของตัวเอง เก็บ remaining_minutes และ expired_at แยกต่อแถว → ลูกค้าซื้อแพ็กเกจเดิมซ้ำได้ไม่จำกัด

5. Single Table Inheritance สำหรับ Role

แทนที่จะแยกตาราง admin/staff/customer ออกจากกัน ระบบเก็บทุก role ในตาราง `users` เดียว โดยใช้คอลัมน์ `role` แยกสิทธิ์ผ่าน Laravel Middleware

role = admin

- จัดการผู้ใช้ / เติมเงิน
- จัดการโซน / แพ็กเกจ / สินค้า
- ดูรายงานทั้งหมด

role = staff

- เปิด/ปิดเครื่องให้ลูกค้า
- จัดการออเดอร์อาหาร
- ดู Seat Map

role = customer

- เลือกที่นั่ง / เปิดเครื่อง
- ซื้อแพ็กเกจ / สั่งอาหาร
- ดูประวัติตัวเอง

6 กฎเกณฑ์ทางธุรกิจ (Business Rules)

กระเป๋าเงิน

- เติมขั้นต่ำ: **20 บาท**
- เติมสูงสุดต่อครั้ง: **10,000 บาท**
- ยอดขั้นต่ำก่อนเปิดเครื่อง: **20 บาท**
- ต้องมียอดพอก่อนสั่งอาหาร

ราคาต่อชั่วโมง

- Standard Zone (165Hz): **40 บาท**
- VIP High-End (240Hz): **60 บาท**
- Duo Private Room: **100 บาท**

บัญชีทดสอบ

Username	Role	Password	Balance
admin	Admin	password	1,000 ฿
staff	Staff	password	500 ฿
customer1	Customer	1	—
customer2	Customer	password	—

ตารางระบบ Laravel

- `migrations` — ประวัติ Migration
- `sessions` — สถานะล็อกอิน
- `password_reset_tokens`
- `cache` / `cache_locks`
- `jobs` / `failed_jobs`